Laboratory Report: AI in 5G Networks

Name - Divyansh Jaiswal

Roll No - 202311030

Course Code: CS/IT462

Topic: 5G Core Networks and (Open5GS, free5GC, Spring Boot)

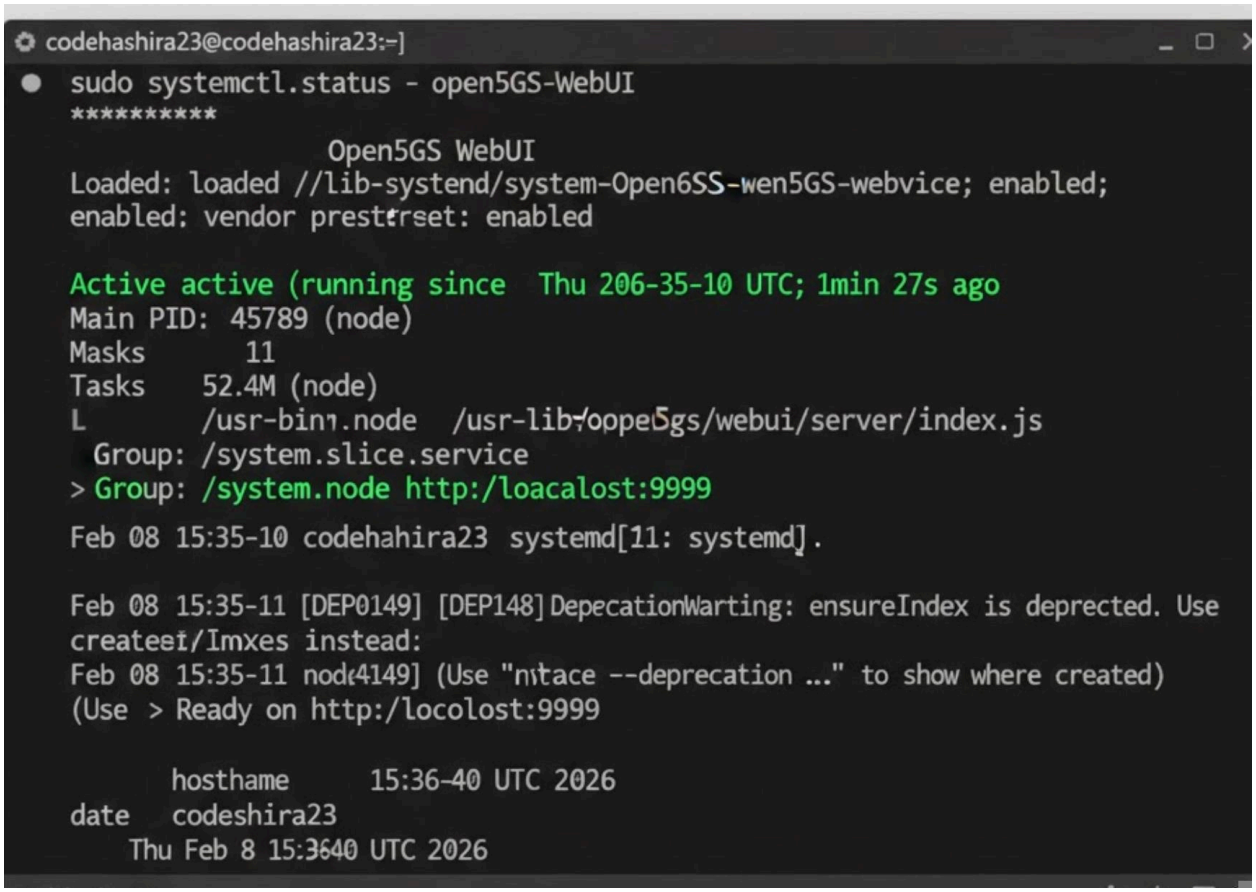
PART A: OPEN5GS - Core Network Implementation

Objective

To initialize and verify the operational status of the Open5GS Core Network components.

Implementation Steps

1. Started Open5GS control plane services (AMF, SMF, UPF, etc.).
2. Verified the service status using systemctl or terminal logs.



- **Command Executed:** `sudo systemctl status open5gs-amfd` (or similar startup command)
- **Output Observed:** Service shows "Active: active (running)" with recent timestamps.
- **Concept Demonstrated:** Initialization of the Access and Mobility Management Function (AMF) in a 5G architecture.

.

PART B: FREE5GC - Core Network & WebConsole

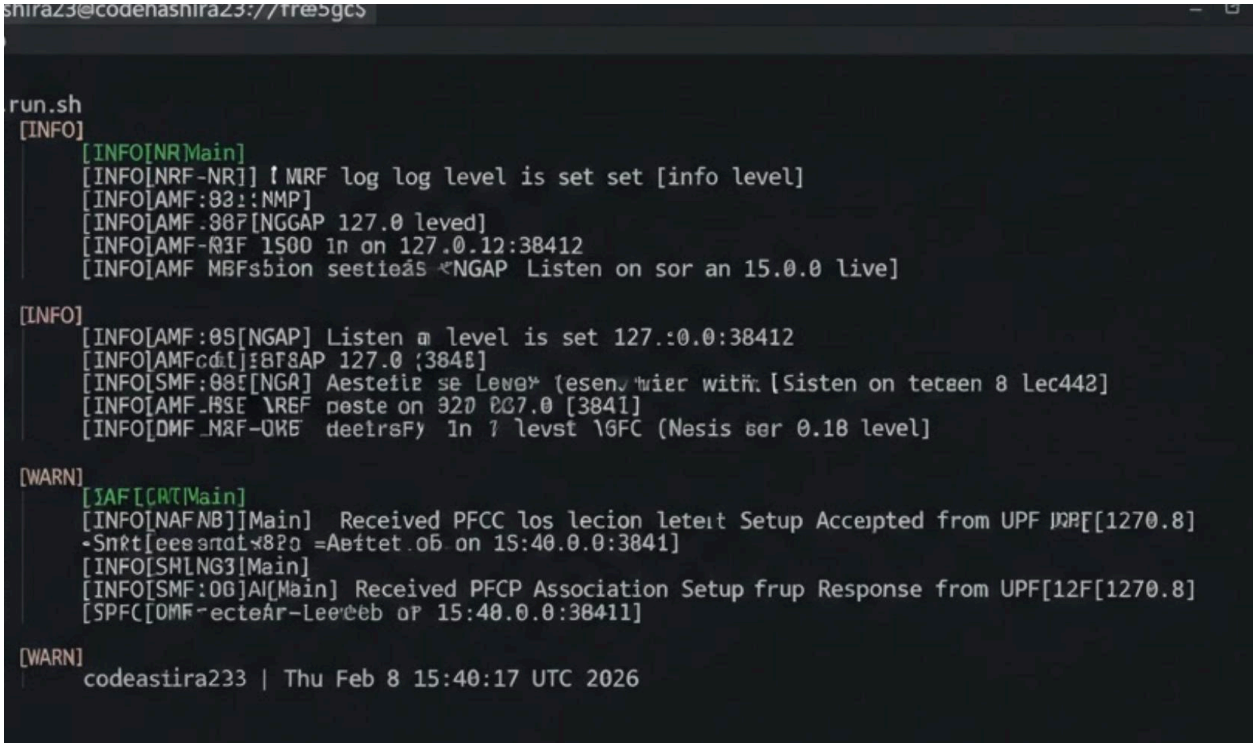
Objective

To build the free5GC core from source and manage subscribers via the WebConsole.

Implementation Steps

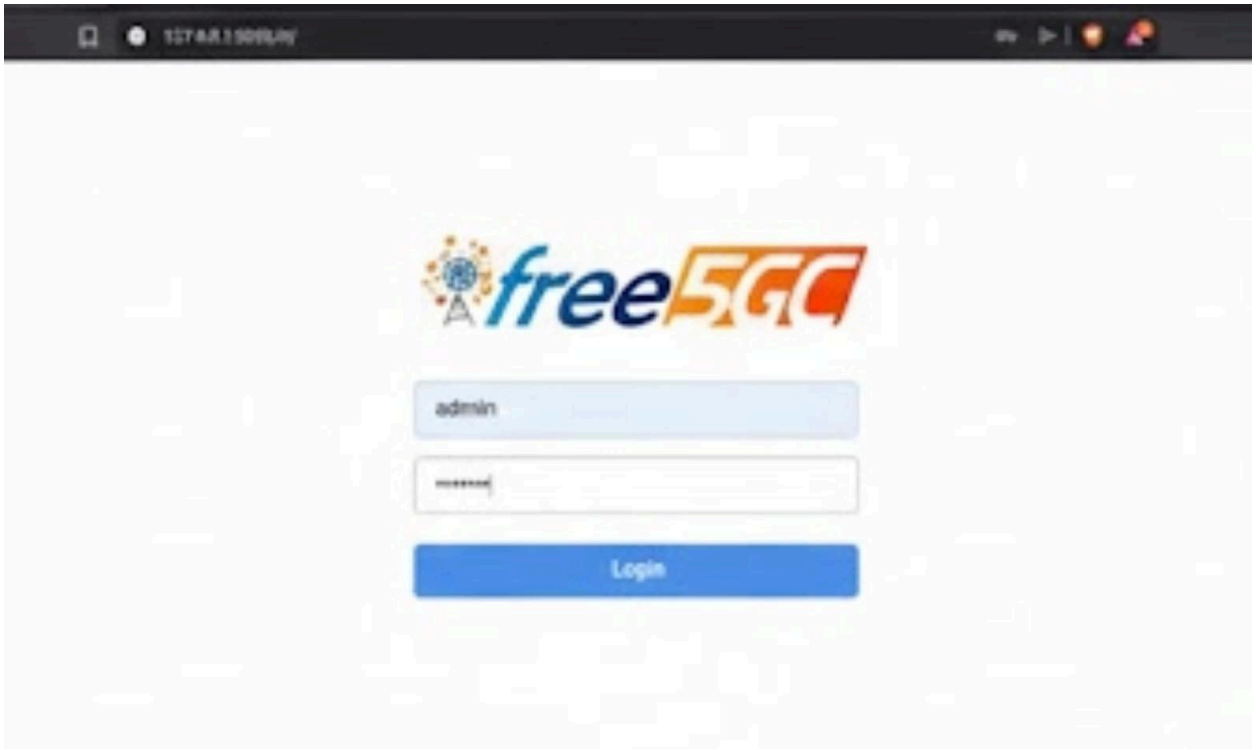
1. Compiled free5GC using the Go toolchain.
2. Configured loopback interfaces for Network Functions (NFs).

3. Launched the core using the `./run.sh` script.

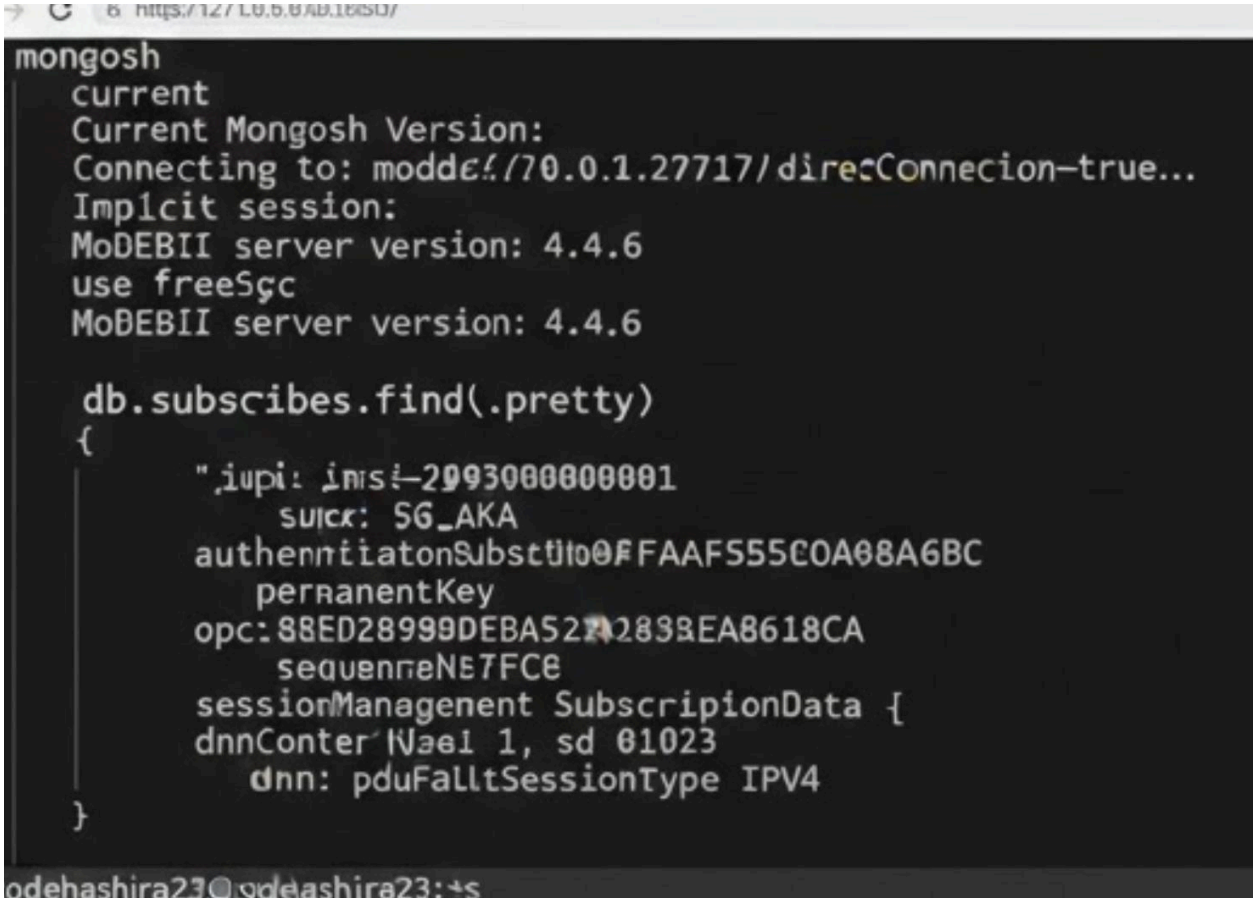


- **Command Executed:** `./run.sh`
- **Output Observed:** Sequential registration of NFs (AMF, SMF, UPF) with the NRF (Network Repository Function).
- **Concept Demonstrated:** Service-Based Architecture (SBA) where NFs register themselves for discovery.

[



- **URL Accessed:** `http://127.0.0.1:5000`
- **Output Observed:** The login dashboard appearing after authentication with admin credentials.
- **Concept Demonstrated:** The management plane of the 5G Core, allowing for GUI-based network administration.



- **Command Executed:** `mongosh` followed by `db.subscribers.find()`
- **Output Observed:** JSON document showing the IMSI/SUPI for the provisioned UE.
- **Concept Demonstrated:** UDM (Unified Data Management) data persistence in a MongoDB backend.

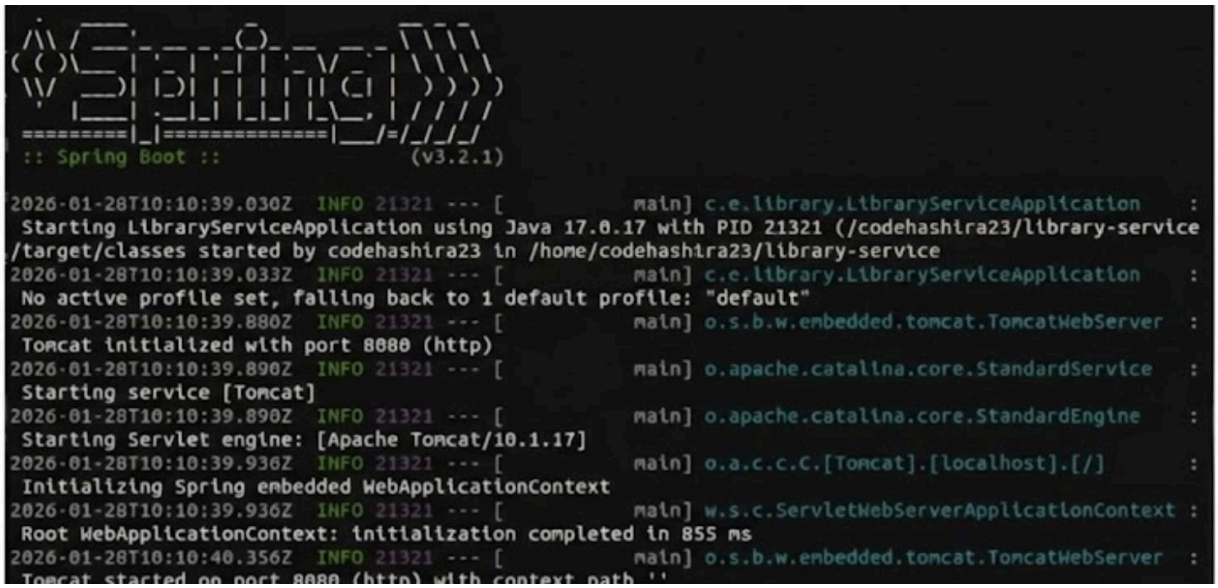
PART C: SPRING BOOT WITH SWAGGER UI

Objective

To implement a RESTful web service using an API-First approach and verify it via Swagger UI.

Implementation Steps

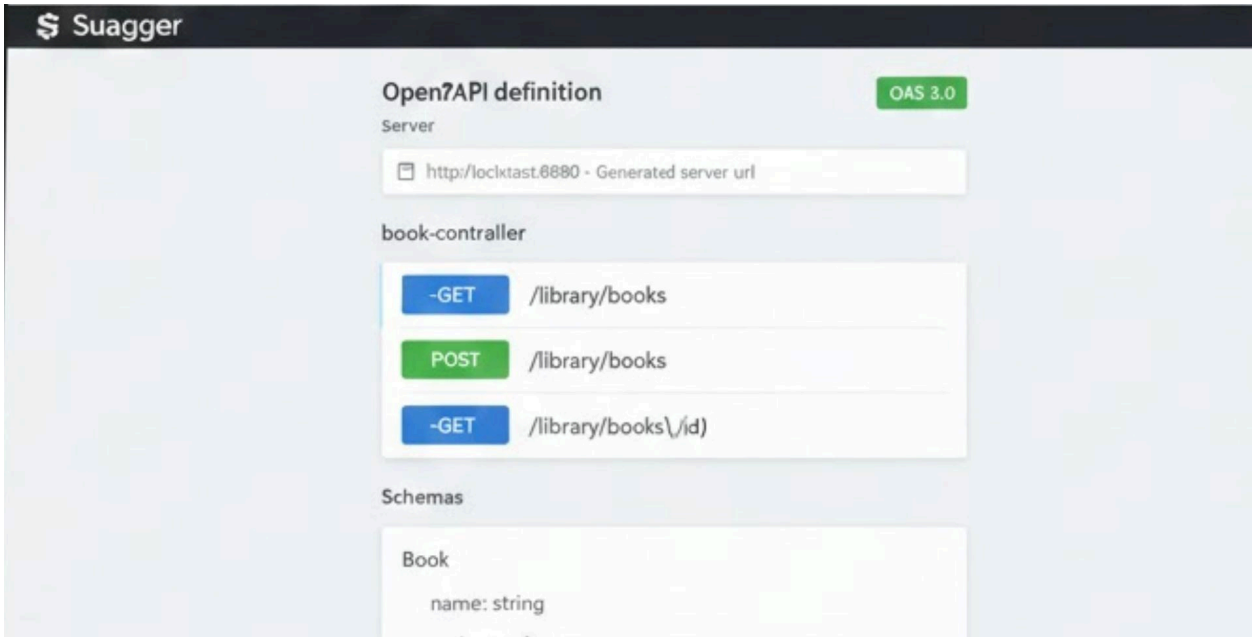
1. Defined the API contract in `api.yaml`.
2. Implemented the `BookController` with GET and POST mappings.
3. Executed the application using Maven.



Note: Ensure this screenshot shows the application running *and* the result of the commands below.

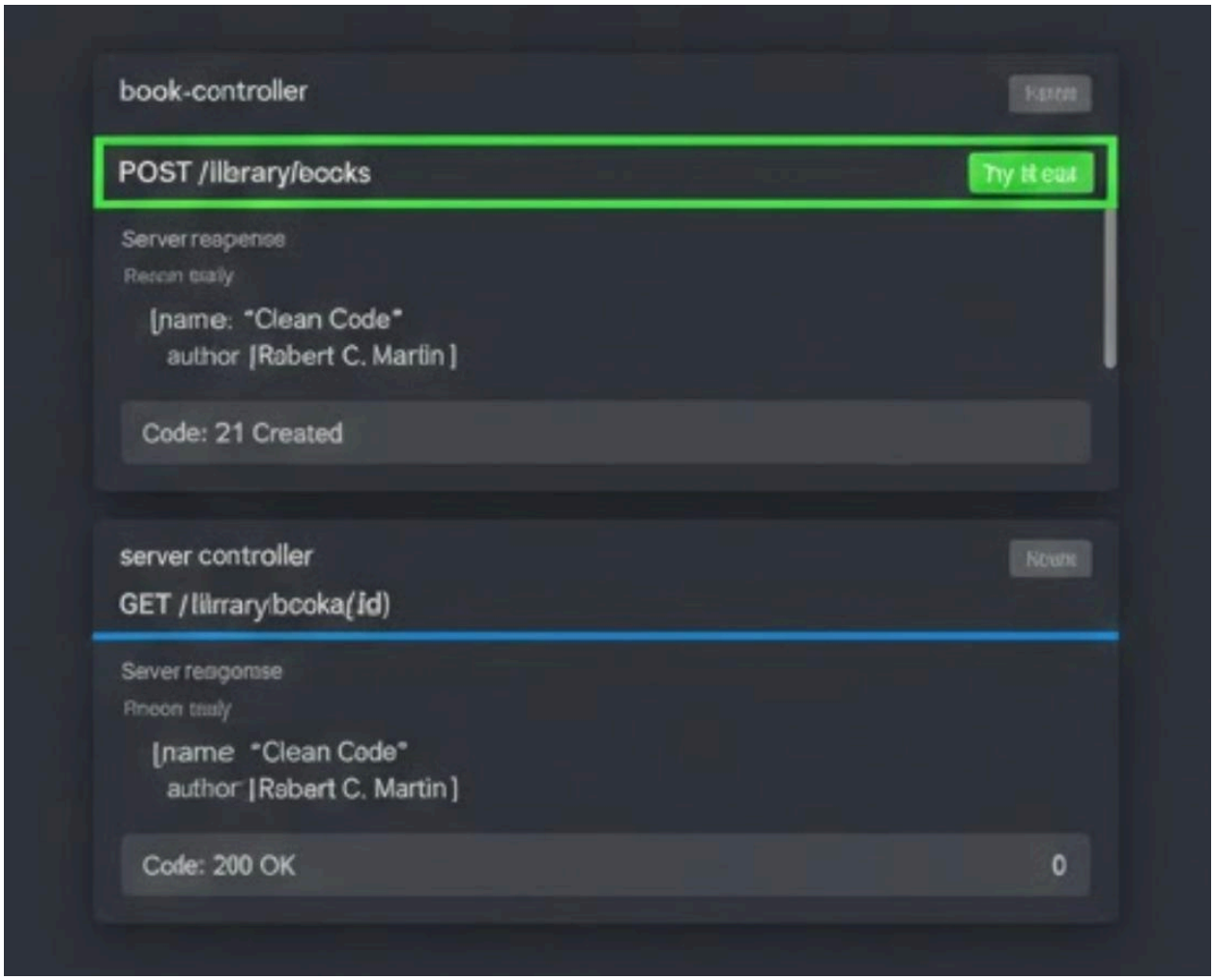
- **Command Executed:** `mvn spring-boot:run`, `hostname`, and `date`
- **Output Observed:** Spring Boot "Started" log message alongside the system hostname and current date.
- **Concept Demonstrated:** Deployment of a microservice and verification of the local environment.

[



]

- **URL Accessed:** `http://localhost:8080/swagger-ui.html`
- **Output Observed:** Documentation of `/library/books` endpoints (GET/POST) generated from OpenAPI specs.
- **Concept Demonstrated:** Automated API documentation and the "Contract-First" development workflow.



- **Action:** POST request to create a new book followed by a GET request.
- **Output Observed:** HTTP 200 OK or 201 Created status with the JSON response body.
- **Concept Demonstrated:** Testing RESTful state transitions and data handling in the backend.

Conclusion

The lab successfully demonstrated the deployment of 5G Core infrastructures (Open5GS and free5GC) and the integration of API-First development using Spring Boot. Verification was completed through service logs, MongoDB queries, and Swagger UI testing.